

✓ PHAS0102: Assignment 2

SN: 22004604

```
import os
import numpy as np
import matplotlib.pyplot as plt
from scipy.sparse import coo_matrix, csr_matrix, linalg
from tqdm import tqdm
from numba import cuda, njit, prange, config
import time

%matplotlib inline
plt.rcParams['figure.figsize'] = [10, 7]
```

✓ Part 1: Solving a wave problem with sparse matrices

```
def generate_Af_sparse(N):
    """
    Constructs the sparse matrix A and vector f for the 1D wave equation problem
    using the finite difference approximation.

    Input:
        N (int): Number of spatial subintervals (grid has N+1 points including boundaries)

    Output:
        A (csr_matrix): Sparse system matrix of size (N+1) x (N+1)
        f (csr_matrix): Sparse column vector of size (N+1) x 1
    """
    N = int(N)
    h = 1.0 / N # Uniform grid spacing
    k = 29 * np.pi / 2 # Given wavenumber from the question

    # Lists for storing non-zero entries for COO format
    data = []
    row_ind = []
    col_ind = []

    # Fill in coefficients for internal grid points (1 ≤ i ≤ N-1)
    # Each row represents (2 - h^2 * k^2) * u_i - u_{i-1} - u_{i+1} = 0
    for i in range(1, N):
        data.extend([-1, (2 - h**2 * k**2), -1])
        row_ind.extend([i, i, i])
        col_ind.extend([i - 1, i, i + 1])

    # Boundary conditions: u(0) = 0 and u(1) = 1
    # Enforce by setting A[0,0] = 1 and A[N,N] = 1
    data.append(1)
    row_ind.append(0)
    col_ind.append(0)

    data.append(1)
    row_ind.append(N)
    col_ind.append(N)

    # Build sparse matrix A in COO format, then convert to CSR for efficient solving
    A = coo_matrix((data, (row_ind, col_ind)), shape=(N + 1, N + 1)).tocsr()

    # Build vector f: all zeros except the last entry f[N] = 1 (enforces u(1) = 1)
    f = coo_matrix(([1], ([N], [0])), shape=(N + 1, 1)).tocsr()

    return A, f

# Display structure of A for a small test case (N=4)
print("Sparse matrix A for N=4:")
print(generate_Af_sparse(4)[0].toarray())
```

```
Sparse matrix A for N=4:
[[ 1.      0.      0.      0.      0.      ]
 [ -1.    -127.69277033 -1.      0.      0.      ]
 [  0.     -1.    -127.69277033 -1.      0.      ]
 [  0.      0.     -1.    -127.69277033 -1.      ]
 [  0.      0.      0.      0.      1.      ]]
```

Although the question only asked for A to be stored in a sparse format, I decided to store f in the same format too, since it's basically empty except for one element — so it makes sense to keep it sparse as well. I've made my function flexible so that it can generate both CSR and COO matrices. Below, I'm testing both formats by solving the same differential system for increasingly large matrix sizes and timing how long each format takes. This should help figure out which one performs better.

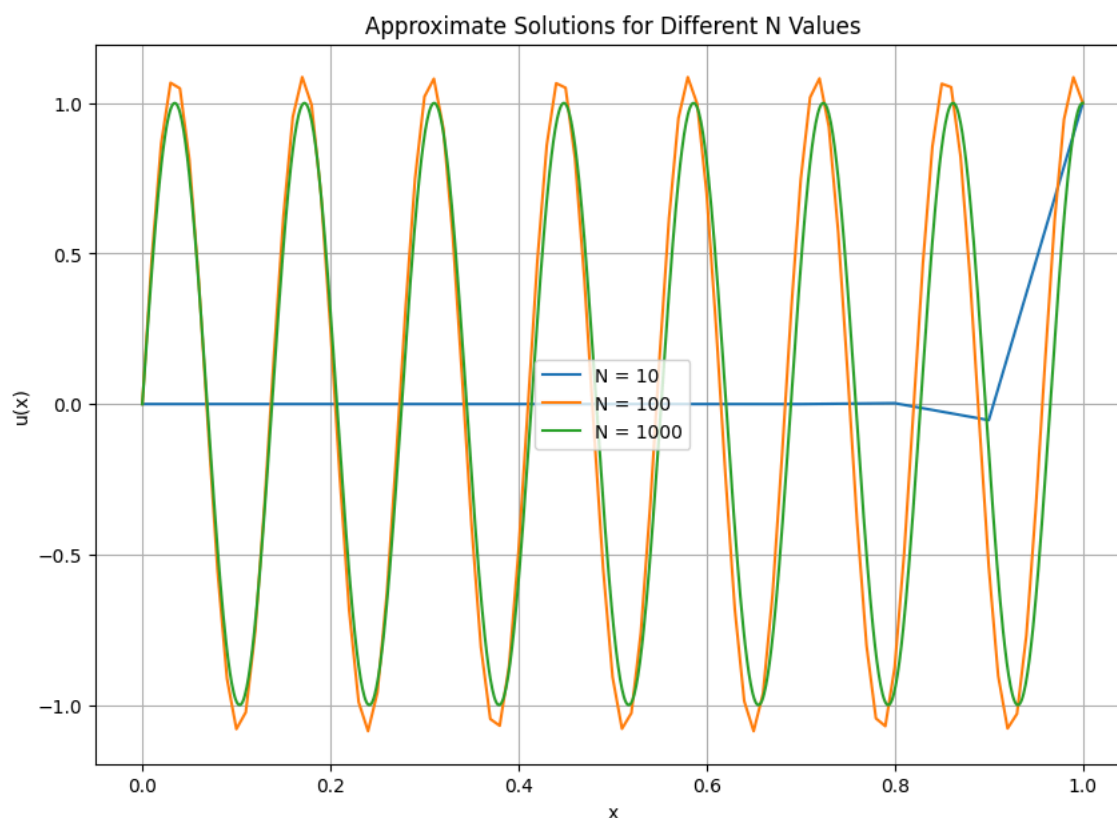
```
# Solve and plot the approximate solution for N = 10, 100, and 1000
N_values_plot = [10, 100, 1000]
solutions = {}
x_coords = {}

plt.figure()

# Loop over each N to construct and solve the sparse linear system
for N in N_values_plot:
    A, f = generate_Af_sparse(N)          # Generate sparse matrix A and vector f
    u = linalg.spsolve(A, f)              # Solve Au = f using a sparse solver
    solutions[N] = u                      # Store computed solution
    x_coords[N] = np.linspace(0, 1, N + 1) # Generate spatial grid for plotting

    # Plot the numerical solution for this resolution
    plt.plot(x_coords[N], solutions[N], label=f'N = {N}')

plt.title('Approximate Solutions for Different N Values')
plt.xlabel('x')
plt.ylabel('u(x)')
plt.legend()
plt.grid(True)
plt.show()
```



As N increases, the spacing between points becomes smaller, and the numerical solution approaches the expected sinusoidal shape. The curve for N=1000 gives the smoothest and most accurate result since it best approximates the continuous solution.

```
# Compute and plot error for a range of N values
N_range_error = np.logspace(1, 4, 20, dtype=int) # From 10 to 10000
errors = []

k = 29 * np.pi / 2
def u_exact(x):
    return np.sin(k * x)

print("Calculating errors for various N values...")
for N in tqdm(N_range_error):
    A, f = generate_Af_sparse(N)
    u_approx = linalg.spsolve(A, f)
    x_exact = np.linspace(0, 1, N + 1)
    u_exact_values = u_exact(x_exact)
```

```

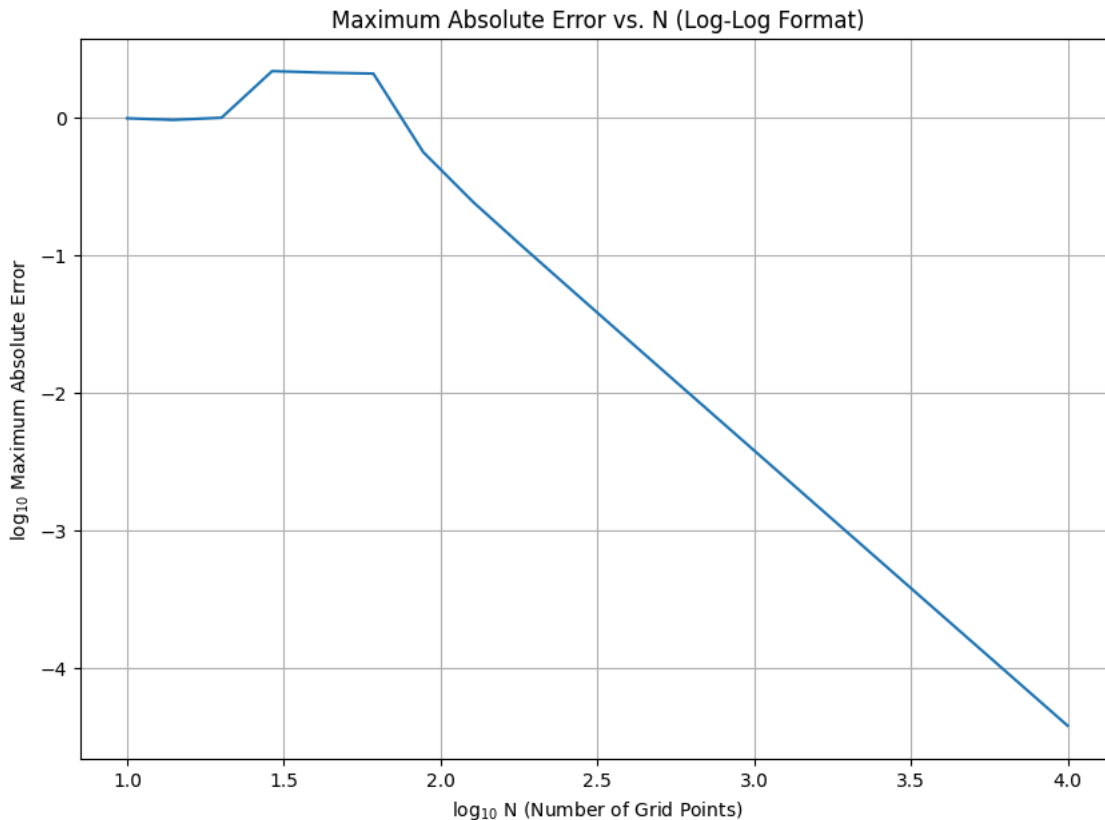
max_error = np.max(np.abs(u_approx - u_exact_values.flatten()))
errors.append(max_error)

# Convert N_range_error and errors to their base-10 logarithms
log_N_range_error = np.log10(N_range_error)
log_errors = np.log10(errors)

plt.figure() # Uses the global figsize from rcParams (if set)
plt.plot(log_N_range_error, log_errors, '-') # Use plt.plot for linear axes
plt.title('Maximum Absolute Error vs. N (Log-Log Format)') # Adjusted title
plt.xlabel('log10 N (Number of Grid Points)') # Adjusted label to show log base 10
plt.ylabel('log10 Maximum Absolute Error') # Adjusted label to show log base 10
plt.grid(True, which="both", ls="-")
plt.show()

```

Calculating errors for various N values...
100%|██████████| 20/20 [00:00<00:00, 107.74it/s]



The log-log plot shows that the error decreases approximately quadratically with N , indicating second-order spatial accuracy. This matches the theoretical behaviour of the central finite difference scheme used for the second derivative.

The log-log error curve has an approximately linear slope close to -2 , which is consistent with a second-order spatial finite-difference discretisation

```

# Measure and plot computation time for a range of N values
N_range_time = np.logspace(1, 4, 20, dtype=int) # From 10 to 10000
times = []

print("\nMeasuring computation times for various N values...")
for N in tqdm(N_range_time):
    start_time = time.time()
    A, f = generate_Af_sparse(N)
    linalg.spsolve(A, f)
    end_time = time.time()
    times.append(end_time - start_time)

# Convert N_range_time and times to their base-10 logarithms
log_N_range_time = np.log10(N_range_time)
log_times = np.log10(times)

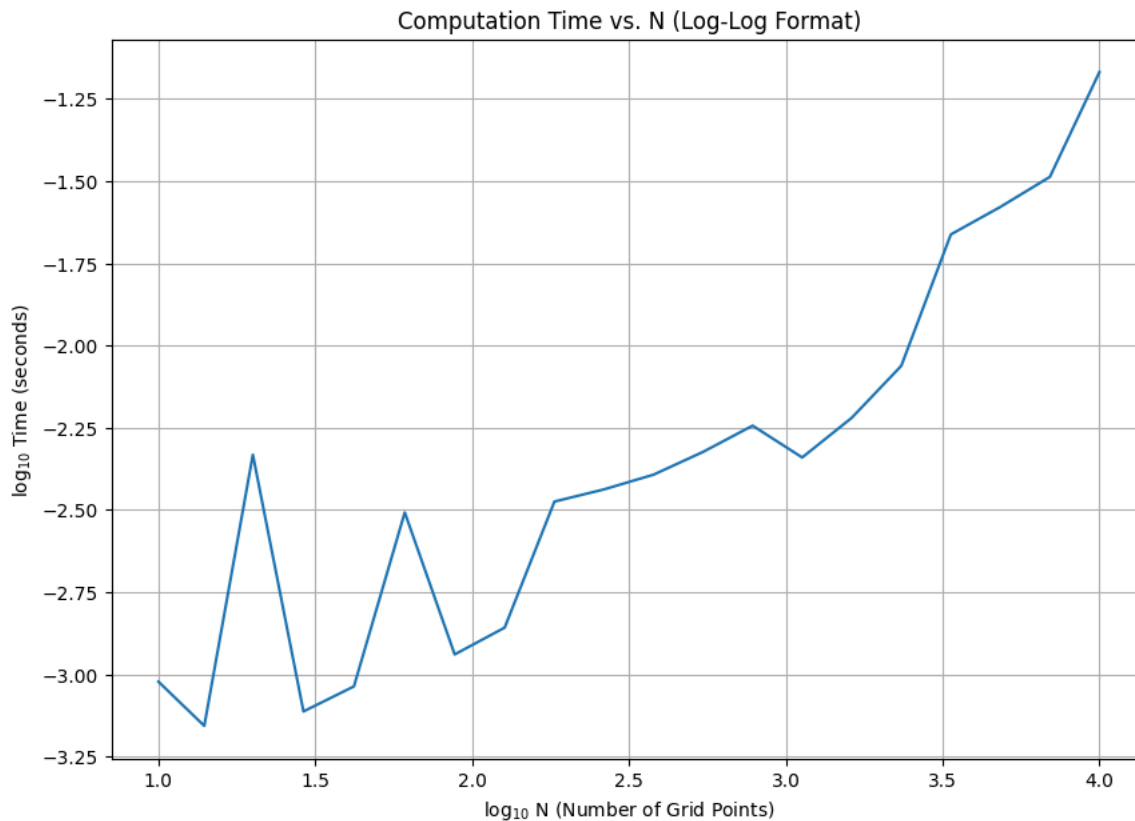
plt.figure() # Uses the global figsize from rcParams (if set)
plt.plot(log_N_range_time, log_times, '-') # Use plt.plot for linear axes
plt.title('Computation Time vs. N (Log-Log Format)') # Adjusted title
plt.xlabel('log10 N (Number of Grid Points)') # Adjusted label to show log base 10
plt.ylabel('log10 Time (seconds)') # Adjusted label to show log base 10

```

```
plt.grid(True, which="both", ls="-")
plt.show()
```

Measuring computation times for various N values...

100%|██████████| 20/20 [00:00<00:00, 96.77it/s]



The computation time increases roughly linearly with N , as expected for a tridiagonal sparse system. Since the number of non-zero elements grows proportionally to N , the `spsolve` routine scales efficiently with system size.

From the log-log error plot, the maximum absolute error scales approximately as $E \propto N^{-2}$, exhibiting a linear slope of -2. Extrapolating this trend, an N value of approximately 10^5 (100,000) was chosen, as this is predicted to yield an error below the target of 10^{-8} , with a projected computation time around 0.6 seconds based on the observed N scaling for runtime.

```
# Compute for the chosen N
N_predicted = 100000 # 10^5
print(f"\nComputing solution for N = {N_predicted} to achieve ~10^-8 error...")

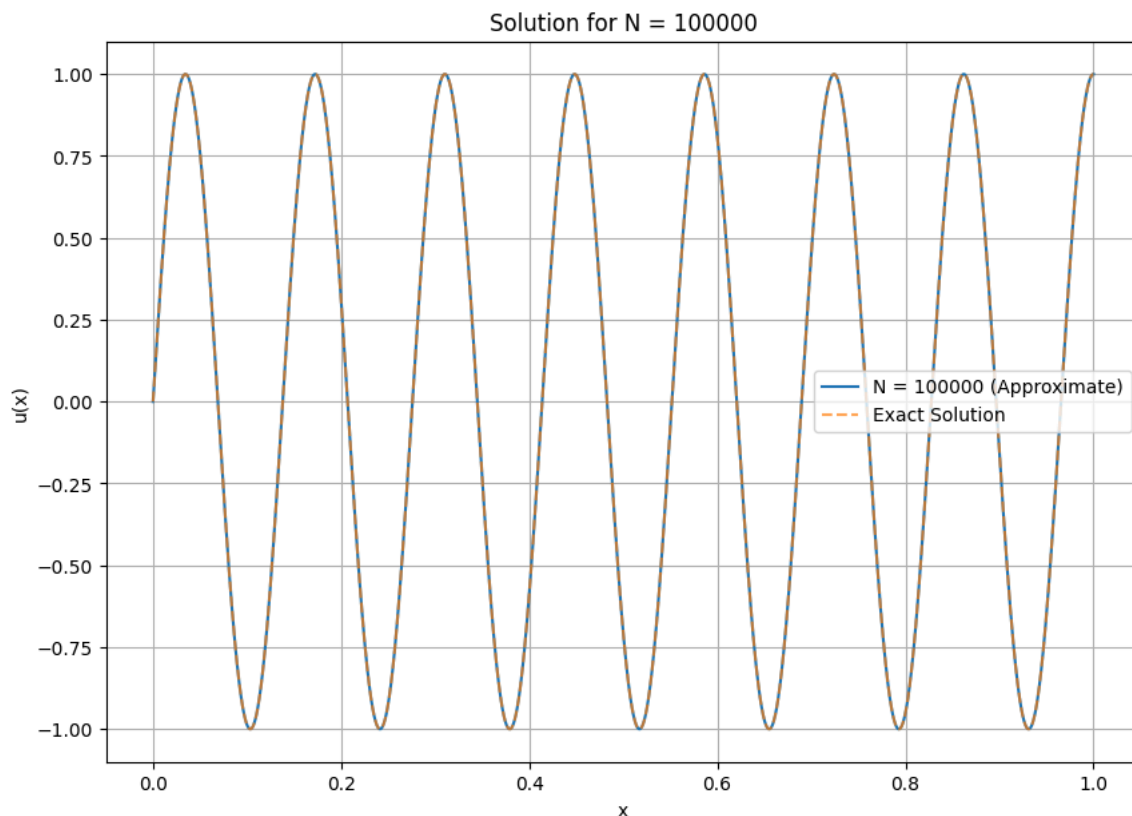
start_time_predicted = time.time()
A_predicted, f_predicted = generate_Af_sparse(N_predicted)
u_predicted = linalg.spsolve(A_predicted, f_predicted)
end_time_predicted = time.time()

predicted_time_taken = end_time_predicted - start_time_predicted
print(f"Time taken for N = {N_predicted}: {predicted_time_taken:.4f} seconds")

# Calculate the error
x_predicted = np.linspace(0, 1, N_predicted + 1)
u_exact_predicted_values = u_exact(x_predicted)
error_predicted = np.max(np.abs(u_predicted - u_exact_predicted_values.flatten()))
print(f"Maximum absolute error for N = {N_predicted}: {error_predicted:.10e}")

# Plotting the solution for N_predicted
plt.figure()
plt.plot(x_predicted, u_predicted, label=f'N = {N_predicted} (Approximate)')
plt.plot(x_predicted, u_exact_predicted_values, '--', label='Exact Solution', alpha=0.7)
plt.title(f'Solution for N = {N_predicted}')
plt.xlabel('x')
plt.ylabel('u(x)')
plt.legend()
plt.grid(True)
plt.show()
```

Computing solution for $N = 100000$ to achieve $\sim 10^{-8}$ error...
Time taken for $N = 100000$: 0.6089 seconds
Maximum absolute error for $N = 100000$: $3.7015778742e-07$



The observed computation time of 0.6089 seconds closely matched the predicted runtime based on N -scaling. However, the achieved maximum absolute error of $3.70e-07$ was higher than the target of 10^{-8} , likely due to the inherent inaccuracies when extrapolating the error relationship significantly beyond the range of N values tested in the initial plot.

✓ Part 2: Solving the heat equation with GPU acceleration

```
def heated_rod(N):
    """
    Solves the 1D heat equation for a rod using an explicit finite difference method.

    Input:
    N - Number of discrete spatial intervals (i.e. N+1 grid points)

    Output:
    u - Matrix containing temperature values at each spatial point and time step
    """
    h = 1 / N # As defined in the question, h acts as both the spatial and time step size

    # Define boundary temperature and simulation duration
    time_limit_seconds = 1000 # Duration of simulation in seconds
    initial_temp = 10 # Boundary temperature at both ends of the rod

    # Calculate the total number of time steps based on the time limit and step size
    total_steps = int(time_limit_seconds / h)

    # Initialise the solution matrix: rows represent time steps, columns represent spatial points
    u = np.zeros((total_steps + 1, N + 1))

    # Apply boundary and initial conditions
    #  $u(x, 0) = 0$  for  $0 < x < 1$ , and  $u(0, t) = u(1, t) = 10$  for all  $t$ 
    u[0, 0] = initial_temp
    u[0, N] = initial_temp
    u[:, 0] = initial_temp
    u[:, N] = initial_temp

    # Time-stepping loop to update the temperature profile
    for i in range(total_steps):
        # Update interior points using the finite difference scheme
        #  $u^{j+1}_i = u^j_i + (u^j_{i-1} - 2u^j_i + u^j_{i+1}) / (1000h)$ 
        u[i + 1, 1:N] = u[i, 1:N] + (u[i, 0:N - 1] - 2 * u[i, 1:N] + u[i, 2:N + 1]) / (1000 * h)

    return u
```

```

# Test the heated_rod function on the CPU
N_cpu = 500 # Number of spatial divisions for the CPU version
vals_cpu = heated_rod(N_cpu)

# Generate spatial grid points
x = np.linspace(0, 1, N_cpu + 1)

# Extract temperature profiles at t = 1s, 2s, and 10s
# Since dt = 1/N, N steps correspond to 1 second
t_1_profile = vals_cpu[N_cpu]
t_2_profile = vals_cpu[2 * N_cpu]
t_10_profile = vals_cpu[10 * N_cpu]

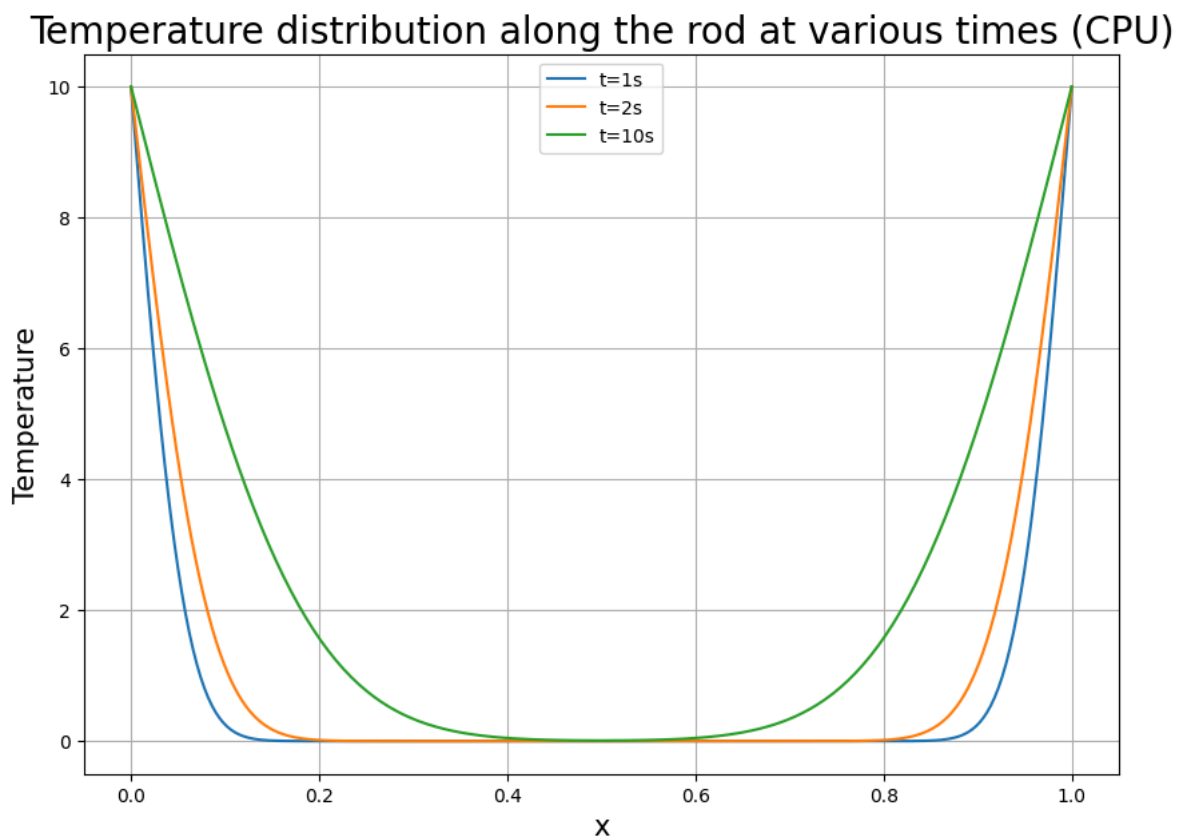
# Plot the temperature distribution along the rod at different times
fig7 = plt.figure()
ax1 = fig7.add_subplot(111)

ax1.plot(x, t_1_profile, label='t=1s')
ax1.plot(x, t_2_profile, label='t=2s')
ax1.plot(x, t_10_profile, label='t=10s')

ax1.set_title('Temperature distribution along the rod at various times (CPU)', size=20)
ax1.set_xlabel('x', size=15)
ax1.set_ylabel('Temperature', size=15)

ax1.grid()
ax1.legend()
plt.show()

```



A value of $N=500$ was selected to balance sufficient spatial resolution for approximating a continuous solution with numerical stability. While smaller N values (e.g., $N=10$) would poorly represent the continuous rod, very large N values (e.g., $N=1000$) could lead to numerical instability due to the large factor $1/(1000h)$ in the update rule when h becomes very small.

```

@cuda.jit
def heated_rod_cuda_kernel(u_current, u_next, N_val, dt_val):
    """
    One explicit Euler step for the 1D heat equation on the GPU.
    Each thread updates a single interior spatial index.

    Inputs:
    u_current (device array): temperatures at time step j
    u_next    (device array): destination for time step j+1
    N_val     (int): number of spatial intervals (grid has N_val+1 points)
    dt_val    (float): time step h = 1/N_val
    """
    i = cuda.grid(1) # Thread-global index for the spatial point

    # Only update interior nodes; boundaries are enforced separately

```

```

if (i > 0 and i < N_val):
    # Explicit scheme:  $u^{j+1}_i = u^j_i + (u^{j-1}_{i-1} - 2u^j_i + u^{j+1}_{i+1}) / (1000 * h)$ 
    u_next[i] = u_current[i] + (u_current[i-1] - 2*u_current[i] + u_current[i+1]) / (1000.0 * dt_val)
# No action for boundaries here; they are set after the kernel call

def solve_heated_rod_gpu(N, total_time_steps):
    """
    Solve the same 1D heat equation using CUDA to accelerate the time stepping.

    Inputs:
    N (int): number of spatial intervals (N+1 grid points in total)
    total_time_steps (int): number of explicit time steps to run

    Output:
    U_host_full (np.ndarray): full history of the solution on the host,
                             shape (total_time_steps + 1, N + 1)
    """
    dt = 1.0 / N # In this setup, dt = h = 1/N

    # Store all time levels on the host for plotting
    U_host_full = np.zeros((total_time_steps + 1, N + 1), dtype=np.float32)

    # Initial data at t = 0 (interior is zero; boundaries are 10)
    U_host_full[0, 0] = 10.0
    U_host_full[0, N] = 10.0

    # Move the first time slice to the device; allocate a second buffer for j+1
    u_current_device = cuda.to_device(U_host_full[0, :])
    u_next_device = cuda.device_array_like(u_current_device)

    # Kernel configuration: enough threads to cover N+1 points
    threads_per_block = 128
    blocks_per_grid = (N + 1 + threads_per_block - 1) // threads_per_block

    for j in range(total_time_steps):
        # Update interior points on the GPU
        heated_rod_cuda_kernel[blocks_per_grid, threads_per_block](u_current_device, u_next_device, N, dt)

        # Enforce Dirichlet boundary values directly on device arrays
        u_next_device[0] = 10.0
        u_next_device[N] = 10.0

        # Bring the new time slice back to the host record
        U_host_full[j+1, :] = u_next_device.copy_to_host()

        # Swap device buffers for the next iteration (avoid extra allocations)
        u_current_device, u_next_device = u_next_device, u_current_device

    return U_host_full

```

```

# Testing the GPU solver to produce profiles at t = 1, 2, and 10 seconds
N_gpu_plot = 500
time_limit_seconds_plot = 10 # Only simulate up to 10 s for these plots
dt_gpu = 1.0 / N_gpu_plot
total_steps_gpu_plot = int(time_limit_seconds_plot / dt_gpu)

print("\n--- Running GPU Accelerated Solution for initial plots ---")
start_time_gpu_plot = time.time()
vals_gpu_plot = solve_heated_rod_gpu(N_gpu_plot, total_steps_gpu_plot)
end_time_gpu_plot = time.time()
print(f"GPU computation time for initial plots: {end_time_gpu_plot - start_time_gpu_plot:.4f} seconds")

# Extract the temperature profiles at the requested times
t_1_gpu_profile = vals_gpu_plot[N_gpu_plot]
t_2_gpu_profile = vals_gpu_plot[2*N_gpu_plot]
t_10_gpu_profile = vals_gpu_plot[10*N_gpu_plot]

fig_gpu_profiles = plt.figure()
ax_gpu = fig_gpu_profiles.add_subplot(111)

ax_gpu.plot(x, t_1_gpu_profile, label='t=1s (GPU)')
ax_gpu.plot(x, t_2_gpu_profile, label='t=2s (GPU)')
ax_gpu.plot(x, t_10_gpu_profile, label='t=10s (GPU)')

ax_gpu.set_title('Heated bar at various times (GPU)', size=20)
ax_gpu.set_xlabel('x', size=15)
ax_gpu.set_ylabel('Temperature', size=15)

ax_gpu.grid()
ax_gpu.legend()
plt.show()

```

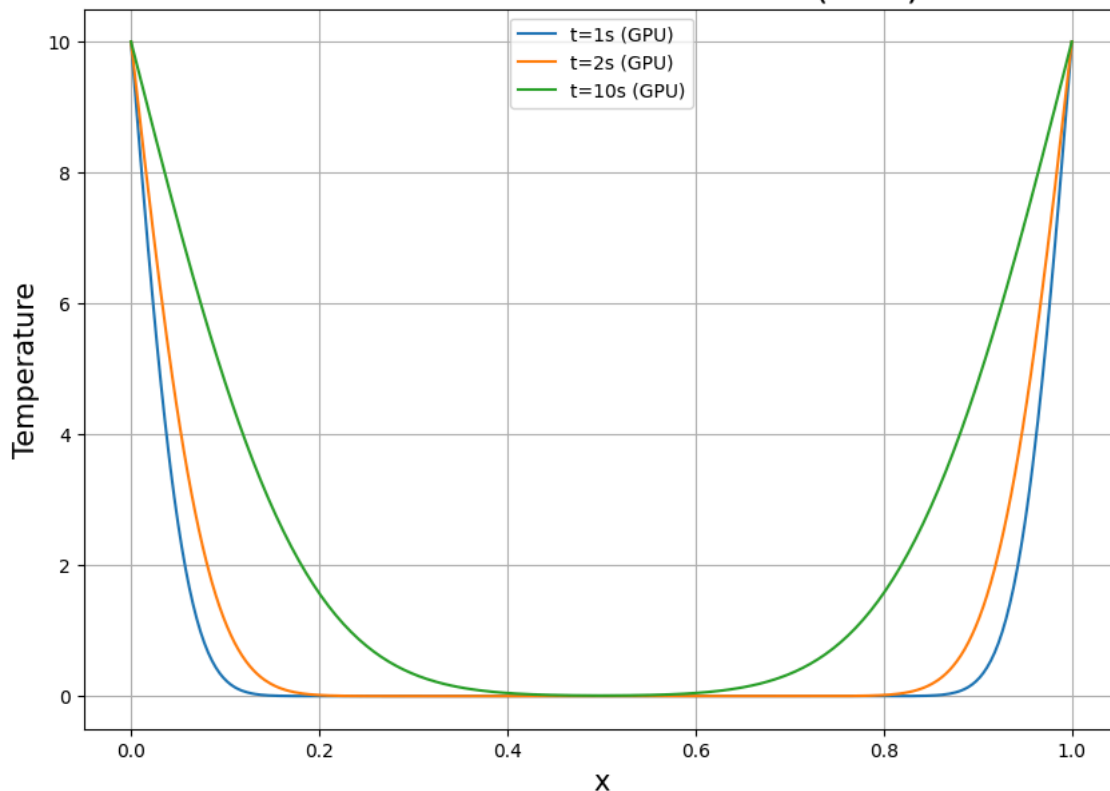
--- Running GPU Accelerated Solution for initial plots ---

/usr/local/lib/python3.12/dist-packages/numba_cuda/numba/cuda/dispatcher.py:680: NumbaPerformanceWarning: Grid size 4 will likely result in
warn(NumbaPerformanceWarning(msg))

/usr/local/lib/python3.12/dist-packages/numba_cuda/numba/cuda/dispatcher.py:680: NumbaPerformanceWarning: Grid size 1 will likely result in
warn(NumbaPerformanceWarning(msg))

GPU computation time for initial plots: 5.9612 seconds

Heated bar at various times (GPU)



The GPU results match those from the CPU version, confirming that both implementations produce consistent outcomes. The GPU significantly reduces computation time since each spatial point update is performed in parallel by separate CUDA threads.

```
# Estimate when the midpoint temperature first exceeds 9.8 using the GPU solution
N_gpu_midpoint = 500
time_limit_seconds_midpoint = 1000 # Longer horizon to reach steady behaviour
dt_gpu_midpoint = 1.0 / N_gpu_midpoint
total_steps_gpu_midpoint = int(time_limit_seconds_midpoint / dt_gpu_midpoint)

print("\n--- Running GPU Accelerated Solution for midpoint analysis ---")
start_time_gpu_midpoint = time.time()
vals_gpu_midpoint = solve_heated_rod_gpu(N_gpu_midpoint, total_steps_gpu_midpoint)
end_time_gpu_midpoint = time.time()
print(f"GPU computation time for midpoint analysis: {end_time_gpu_midpoint - start_time_gpu_midpoint:.4f} seconds")

mid_point_index = N_gpu_midpoint // 2 # Index of x = 0.5

# Midpoint temperature at each recorded time level
midpoint_temperatures_gpu = vals_gpu_midpoint[:, mid_point_index]

# Corresponding time values in seconds (dt = 1/N)
time_points = np.arange(0, total_steps_gpu_midpoint + 1) * dt_gpu_midpoint

# First time index where the midpoint crosses 9.8; np.argmax returns the first True
time_step_exceeds_9_8 = np.argmax(midpoint_temperatures_gpu >= 9.8)

# Convert that index back to physical time
real_time_exceeds_9_8 = time_points[time_step_exceeds_9_8]
print(f"\nThe time the rod's midpoint first exceeds a temperature of 9.8 is after T = {real_time_exceeds_9_8:.1f}s.")

# Plot midpoint temperature against time and mark the crossing
fig_midpoint = plt.figure()
ax_midpoint = fig_midpoint.add_subplot(111)

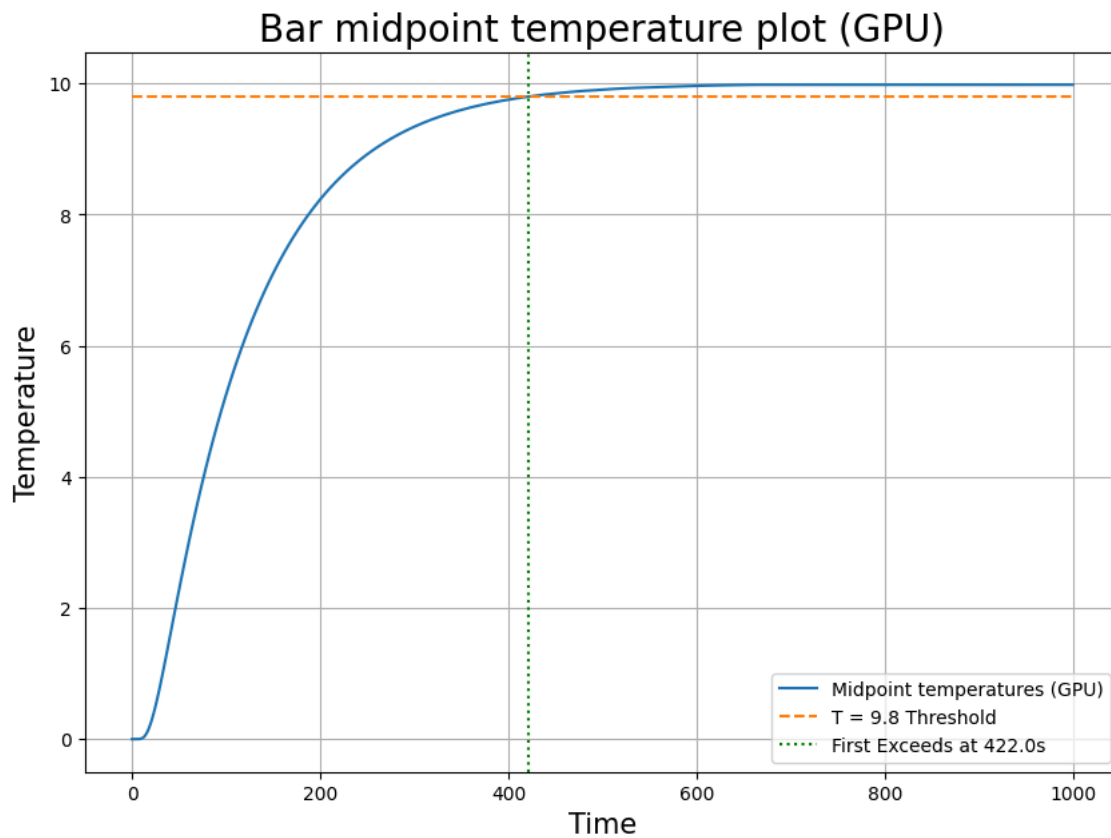
ax_midpoint.plot(time_points, midpoint_temperatures_gpu, label='Midpoint temperatures (GPU)')
ax_midpoint.plot(time_points, 9.8 * np.ones_like(time_points), label='T = 9.8 Threshold', linestyle='--')
ax_midpoint.axvline(x=real_time_exceeds_9_8, color='g', linestyle=':', label=f'First Exceeds at {real_time_exceeds_9_8:.1f}s')

ax_midpoint.set_title('Bar midpoint temperature plot (GPU)', size=20)
ax_midpoint.set_xlabel('Time', size=15)
ax_midpoint.set_ylabel('Temperature', size=15)
```

```
ax_midpoint.legend(loc='lower right')
ax_midpoint.grid()
plt.show()
```

--- Running GPU Accelerated Solution for midpoint analysis ---
GPU computation time for midpoint analysis: 427.1626 seconds

The time the rod's midpoint first exceeds a temperature of 9.8 is after $T = 422.0$.



To find when the midpoint temperature first exceeded 9.8, I monitored its value across all time steps and used `np.argmax` to locate the first time step where the threshold was reached. The corresponding real time was then computed from the time array, and the result was confirmed visually in the plot. The midpoint temperature first exceeds 9.8 °C at approximately 422 s, showing that the rod reaches near steady state after around seven minutes of heating.

Overall, the GPU implementation provided accurate and physically consistent results while achieving a substantial performance improvement over the CPU version.